

AEGIS-X — Complete Technical Documentation

Continuous Behavioral Trust Infrastructure for Indian Banking DFS & IBA Cyber Security PSBs Hackathon 2026 —
Central Bank of India × MNNIT Allahabad

Table of Contents

1. [The Problem We Solve](#)
2. [Our Solution — What AEGIS-X Is](#)
3. [System Architecture](#)
4. [The Trust Pipeline — How It Works](#)
5. [SDK — Data Collection Layer](#)
6. [Backend — ML & Decision Engine](#)
7. [Dashboard — SOC Monitoring Interface](#)
8. [Mathematical Foundation](#)

- 9. [Security & Production Hardening](#)
- 10. [Attack Scenarios & Detection](#)
- 11. [Tech Stack](#)
- 12. [API Reference](#)
- 13. [Deployment](#)
- 14. [What We Built — Complete Feature List](#)

1. The Problem We Solve

The Gap in Indian Banking Security

Every existing banking security mechanism — UPI PIN, OTP, biometric login, device binding — verifies that **the user is present**. None of them verify that **the user is acting freely**.

Real-world attack scenarios that bypass ALL existing protections:

Attack	What Happens	Why Existing Auth Fails
Scam Call (Vishing)	Fraudster calls victim posing as bank officer, instructs them to transfer money "for verification"	User willingly enters their PIN. OTP is received on their phone. Face ID passes.

Remote Access Malware	AnyDesk/TeamViewer installed, attacker controls the phone remotely	Device is the real device. Location is correct. All credentials are entered correctly.
Digital Arrest Scam	Victim told they'll be arrested unless they transfer money immediately	Panicked user completes the transaction themselves — all auth succeeds.

Scale of the problem (India 2024-25): - ■11,333 crore lost to cyber fraud (RBI data) - 70% of UPI fraud involves social engineering where the user *willingly authenticates* - Average victim realizes they've been scammed 15-30 minutes AFTER the transaction

AEGIS-X detects fraud DURING the transaction — not after.

2. Our Solution — What AEGIS-X Is

AEGIS-X is a **continuous behavioral trust engine** that runs silently inside a banking app, monitoring HOW the user interacts with their phone — not WHAT they do.

Core Insight

Every person has a unique "behavioral fingerprint" — their typing rhythm, swipe patterns, touch pressure, hesitation patterns, and phone holding style. This fingerprint changes dramatically under stress, coercion, or when someone else is controlling the device.

What We Do

1. **Collect** — A lightweight SDK (15KB) captures 16 behavioral signals every 2 seconds

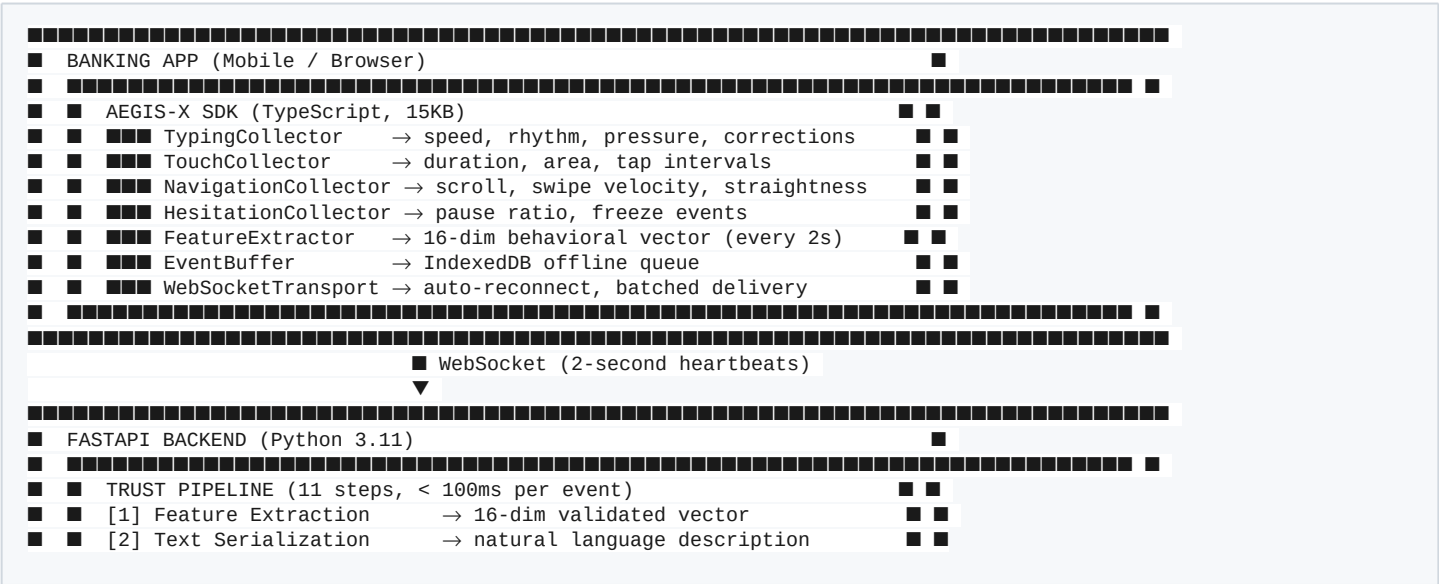
- 2. **Compare** — Each behavioral snapshot is compared against the user's established "identity baseline"
- 3. **Detect** — When behavior deviates (panic, robotic input, different person), we detect it in real-time
- 4. **Decide** — ALLOW (proceed normally), STEP_UP (require additional verification), or BLOCK (halt transaction + alert)
- 5. **Explain** — Every decision comes with a human-readable audit trail

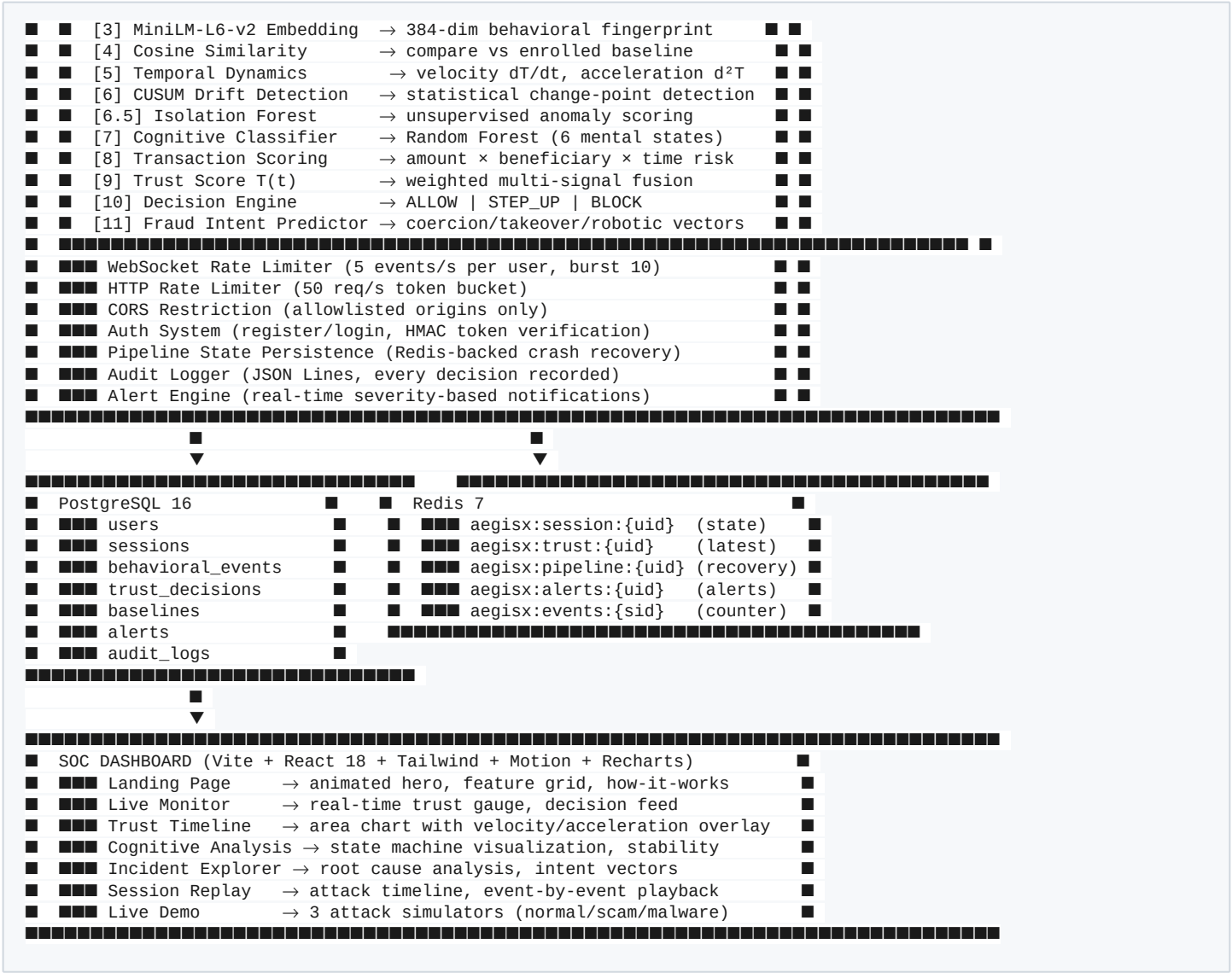
Key Differentiator

"AEGIS-X doesn't verify who you are — it verifies that you're okay."

We don't replace PIN/OTP. We add a **coercion-aware layer** on top that catches the attacks where the user passes all traditional auth but is acting under duress.

3. System Architecture





4. The Trust Pipeline — How It Works

The pipeline executes every 2 seconds per user, completing in under 100ms. Here's what happens at each step:

Step 1: Feature Extraction

Raw SDK telemetry (16 fields) is validated, clipped to physical bounds, and extracted into a canonical feature vector.

The 16 behavioral features:

#	Feature	What It Captures
1	typing_speed_cps	Characters per second typed
2	typing_rhythm_variance	Consistency of inter-key timing
3	typing_pressure_mean	How hard they press keys
4	swipe_velocity_mean	Average swipe speed
5	swipe_velocity_variance	Consistency of swipe speeds
6	swipe_straightness	Linearity of swipe paths (1.0 = perfectly straight)
7	touch_duration_mean	How long finger stays on screen
8	touch_duration_variance	Consistency of touch durations
9	touch_area_mean	Finger contact area (pressure indicator)
10	hesitation_ratio	Fraction of 2s window spent idle
11	hesitation_count	Number of distinct pause events
12	correction_rate	Backspace/undo frequency
13	scroll_speed_mean	Scrolling velocity
14	gyroscope_variance	Device shake/tremor (hand stability)
15	session_time_elapsed	Time since session start
16	interaction_intensity	Total touch events in 2s window

Step 2: Behavioral Text Serialization

Numeric features are converted to natural language descriptions: - typing_speed_cps: 1.2 → "Typing speed is extremely slow, indicating hesitation or distraction." - gyroscope_variance: 0.08 → "Extreme device shaking, indicating severe physical agitation." - hesitation_ratio: 0.7 → "Extreme hesitation, user is frozen or severely conflicted."

Why text? MiniLM understands semantic similarity. "Panicked typing" and "distressed input" produce similar embeddings — this gives robustness across different devices and users.

Step 3: MiniLM Embedding (384-dim)

The behavioral description is encoded by all-MiniLM-L6-v2 (sentence-transformers) into a 384-dimensional vector — the user's **behavioral fingerprint** for this moment.

- L2-normalized to unit sphere for cosine similarity
- ~55ms inference on CPU (the pipeline bottleneck)
- Singleton model — loaded once, reused across all users

Step 4: Cosine Similarity vs Baseline

The current embedding is compared against the user's stored behavioral baseline (their "normal" fingerprint, established during enrollment).

- Similarity > 0.90 → TRUSTED (same person, normal behavior)
- Similarity 0.75-0.90 → NORMAL (natural day-to-day variance)
- Similarity 0.60-0.75 → SUSPICIOUS (significant deviation)

- Similarity < 0.60 → HIGH_RISK (different person or extreme state)

Step 5: Temporal Dynamics

History buffer maintains the last 50 similarity scores (100 seconds of data) and computes: - **Trust Velocity** (dT/dt): rate of trust change - **Trust Acceleration** (d^2T/dt^2): is decay speeding up or slowing down? - **Behavioral Entropy** $H(t)$: randomness/oscillation in behavior (social engineering signature)

Step 6: CUSUM Drift Detection

Cumulative Sum (Page, 1954) algorithm detects change-points in the similarity stream: - Accumulates evidence of sustained deviation from expected similarity (0.92) - Fires when accumulated evidence exceeds threshold (0.15) - Also detects instant jumps (single-step drops > 0.10) - Classifies severity: none → low → medium → high → critical

Step 6.5: Isolation Forest Anomaly Detection

Unsupervised anomaly detector on the raw 16-dim feature space: - Pre-seeded with 100 synthetic normal samples (no cold-start gap) - Continuously learns as real data flows in - Catches novel attack patterns without labeled training data - Returns anomaly_score [0,1] where higher = more anomalous

Step 7: Cognitive State Classification

Random Forest classifier predicts the user's mental state from 8 key features:

State	Stability Score	What It Means
-------	-----------------	---------------

calm	1.00	Normal, relaxed interaction
focused	0.85	Concentrated but natural
distressed	0.50	Elevated uncertainty, possible manipulation
panicked	0.30	Severe distress — likely active scam
coerced	0.15	External pressure detected — someone instructing them
robotic	0.05	Automated/scripted — malware or remote access

Features used: hesitation_ratio, correction_rate, typing_speed, typing_rhythm_variance, touch_duration_mean, gyroscope_variance, interaction_intensity, swipe_straightness.

Step 8: Transaction Scoring

Evaluates the pending transaction for risk signals: - **Amount thresholds:** ■5K (low) → ■25K (medium) → ■1L (high) → ■5L (extreme) - **New beneficiary:** 70% of UPI fraud involves first-time recipients - **Time of day:** 2-5 AM transactions are higher risk - **Frequency:** 10+ transactions/day is unusual

Step 9: Trust Score Computation

$$T(t) = 0.40 \times \text{behavioral_similarity}$$

+ 0.20 × device_trust

+ 0.20 × transaction_normality

+ 0.20 × cognitive_stability

$$\text{Effective_T} = T(t) - \text{drift_penalty}$$

Step 10: Decision Engine

Converts trust score to action with override rules:

Condition	Action
T > 0.80	ALLOW
T ∈ [0.50, 0.80]	STEP_UP (verify)
T < 0.50	BLOCK
Cognitive = coerced/robotic	→ instant BLOCK
Rapid decline + ALLOW zone	→ escalate to STEP_UP
Critical drift + STEP_UP	→ escalate to BLOCK
High amount + panicked	→ BLOCK

Step 11: Fraud Intent Prediction

Estimates probability vectors for attack classification: - **Coercion probability**: based on cognitive state + trust decline speed - **Takeover probability**: based on similarity deviation + drift severity - **Robotic probability**: based on entropy + cognitive = robotic - **Anomaly severity**: combined isolation forest + drift signal

5. SDK — Data Collection Layer

Location: sdk/

The SDK is a TypeScript library designed to embed into any banking mobile/web app. It captures behavioral signals passively without affecting user experience.

Architecture

```
 sdk/
■■■ index.ts → Public API: init(), start(), stop(), onDecision()
```

```

■■■ core/
■   ■■■ AegisX.ts           → Main orchestrator class
■   ■■■ collectors/
■     ■■■ TypingCollector.ts → Keystroke dynamics (speed, rhythm, pressure)
■     ■■■ TouchCollector.ts  → Touch events (duration, area, intervals)
■     ■■■ NavigationCollector.ts → Scroll/swipe patterns (velocity, straightness)
■     ■■■ HesitationCollector.ts → Idle detection (pause ratio, freeze events)
■   ■■■ processors/
■     ■■■ FeatureExtractor.ts → Combines collectors → 16-dim vector
■   ■■■ storage/
■     ■■■ EventBuffer.ts     → IndexedDB offline queue (survives network loss)
■   ■■■ transport/
■     ■■■ WebSocketTransport.ts → Auto-reconnect, batched delivery, heartbeat
■■■ browser/
■   ■■■ BrowserSDK.ts        → Browser-specific bindings (keyboard + mouse)

```

How It Works

1. SDK attaches event listeners to the app's UI (passive, non-blocking)
2. Every 2 seconds, FeatureExtractor aggregates raw events into a 16-dim vector
3. Vector is sent via WebSocket to the backend
4. If offline, events queue in IndexedDB and replay when connection restores
5. Backend responds with trust decision — SDK can trigger UI actions (step-up dialog, block screen)

Integration (3 lines)

```

import { AegisX } from '@aegisx/sdk'

const aegis = new AegisX({ userId: 'user_123', endpoint: 'wss://api.bank.com/ws' })
aegis.start()
aegis.onDecision((decision) => {
  if (decision.action === 'BLOCK') showBlockScreen()
  if (decision.action === 'STEP_UP') showBiometricPrompt()
})

```

6. Backend — ML & Decision Engine

Location: backend/

Directory Structure

```
backend/
├── main.py                → FastAPI app, WebSocket endpoints, lifespan
├── api/
│   ├── auth_routes.py    → Register/Login/Verify (HMAC tokens)
│   ├── session_routes.py → Start/End session
│   ├── event_routes.py   → Submit behavioral event (HTTP fallback)
│   ├── monitor_routes.py → Dashboard data queries
│   ├── audit_routes.py   → Explainability & compliance
│   └── dependencies.py   → Lazy-loaded EventProcessor singleton
├── core/
│   ├── config.py         → All configuration (env vars, weights, thresholds)
│   ├── auth.py           → API key verification
│   ├── rate_limiter.py   → Token bucket HTTP rate limiting
│   ├── metrics.py        → Latency tracking (p50/p95/p99)
│   ├── validators.py     → Input validation & sanitization
│   └── logging.py        → Structured JSON logging
├── models/
│   └── database.py        → SQLAlchemy ORM (7 tables)
├── schemas/
│   ├── requests.py       → Pydantic request validation (bounds checking)
│   └── responses.py      → Pydantic response models
├── services/
│   ├── trust_pipeline.py → 11-step orchestrator (THE core engine)
│   ├── event_processor.py → Central controller (WebSocket → Pipeline → Response)
│   ├── session_manager.py → Thin facade for backward compatibility
│   ├── feature_engineering.py → 16-dim extraction + validation
│   ├── serializer.py     → Numeric → natural language descriptions
│   ├── embedding_service.py → MiniLM-L6-v2 (singleton, 384-dim)
│   ├── baseline_service.py → Enrollment, EMA adaptation, anti-poisoning
│   ├── similarity_service.py → Cosine similarity + risk classification
│   ├── history_service.py → Sliding window buffer + temporal dynamics
│   ├── drift_service.py  → CUSUM change-point detector
│   ├── anomaly_service.py → Isolation Forest (pre-seeded, no cold-start)
│   ├── cognitive_service.py → Random Forest cognitive state machine
│   ├── trust_service.py  → T(t) computation + TransactionScorer + DeviceScorer
│   ├── decision_service.py → ALLOW/STEP_UP/BLOCK + escalation rules
│   ├── fraud_predictor.py → Intent vectors + trajectory classification
│   ├── risk_service.py   → Unified risk aggregation
│   ├── explanation_service.py → Incident narratives + root cause analysis
│   └── cache_service.py  → Redis (graceful fallback to in-memory)
└── websocket/
    └── socket_manager.py → Connection registry, SDK + dashboard broadcast
```

Key Design Decisions

1. **Single pipeline, no duplication** — EventProcessor is the sole orchestrator. SessionManager delegates to it.

- 2. **Stateless services, stateful context** — Pipeline services are shared (loaded once). Per-user state lives in PipelineContext.
- 3. **State persistence** — Pipeline state (CUSUM, history) persists to Redis every 5 events for crash recovery.
- 4. **Graceful degradation** — Redis down? Falls back to in-memory. Postgres down? Pipeline still works.
- 5. **Anti-poisoning** — Baseline only updates when trust > 0.90 (attacker can't corrupt the identity).

7. Dashboard — SOC Monitoring Interface

Location: `dashboard/`

A real-time Security Operations Center (SOC) dashboard built with modern React for bank fraud analysts.

Pages

Page	Purpose
Landing Page	Animated hero with card swap, bento grid, ripple effects, flip words
Live Demo	3 attack simulators running in real-time (normal/scam/malware)
Live Monitor	Real-time trust gauge, decision feed, active session list
Trust Timeline	Area chart showing trust trajectory with velocity/acceleration overlays
Cognitive Analysis	State machine visualization, stability metrics, escalation tracking
Incident Explorer	Root cause analysis, fraud intent vectors, recommended actions

Session Replay	Event-by-event playback of attack sessions
Login/Register	Authentication with HMAC token-based sessions

Tech

- **Vite** — instant HMR, optimized builds
- **React 18** — concurrent features, suspense
- **Tailwind CSS 4** — utility-first styling
- **Motion** (Framer Motion) — smooth animations
- **Recharts** — real-time trust charts
- **Three.js** — 3D shader effects on landing
- **WebSocket** — live dashboard updates (broadcast from backend)

Real-Time Data Flow

Backend → WebSocket broadcast → Dashboard store → React components re-render

Every trust decision is pushed to all connected dashboard clients immediately, enabling fraud analysts to see attacks unfold in real-time.

8. Mathematical Foundation

Trust Score Formula

$$T(t) = w_S \cdot S(t) + w_D \cdot D(t) + w_{Tx} \cdot Tx(t) + w_C \cdot C(t)$$

Where:

$S(t)$	= behavioral_similarity (cosine sim vs baseline)	$[w_S = 0.40]$
$D(t)$	= device_trust (known device, location, rooted?)	$[w_D = 0.20]$
$Tx(t)$	= transaction_normality (amount, beneficiary, time)	$[w_{Tx} = 0.20]$
$C(t)$	= cognitive_stability (Random Forest output)	$[w_C = 0.20]$

$$\text{Effective_T} = T(t) - \text{drift_penalty}(\text{severity})$$

Temporal Dynamics

Trust Velocity: $v(t) = dT/dt \approx \text{mean}(\Delta T \text{ over last 5 events})$
Trust Acceleration: $a(t) = d^2T/dt^2 \approx \text{mean}(\Delta v \text{ over last 8 events})$
Behavioral Entropy: $H(t) = \text{normalized_std}(\text{similarity_window}) / 0.1$

- $v < 0 + a < 0 \rightarrow$ **Accelerating decay** (worst case: active attack)
- $v < 0 + a > 0 \rightarrow$ **Decelerating decay** (attack may be stabilizing)
- $v > 0 \rightarrow$ **Recovering** (user returning to normal)
- $H > 0.5 \rightarrow$ **Oscillating** (social engineering signature)

CUSUM Algorithm

$$S_{\mu}(t) = \max(0, S_{\mu}(t-1) + (\mu_{\mu} - x(t) - k))$$

Where:

$\mu_{\mu} = 0.92$ (expected similarity for normal user)
 $k = 0.03$ (allowance - ignore deviations < 3%)

$h = 0.15$ (threshold – cumulative evidence needed to fire)

Drift detected when: $S_{\blacksquare}(t) > h$ OR $|x(t) - x(t-1)| > 0.10$ (instant jump)

Baseline Adaptation (Anti-Poisoning)

$baseline_new = \alpha \cdot baseline_old + (1 - \alpha) \cdot current_embedding$

where:
 $\alpha = 0.95$ (EMA decay – slow adaptation)
Only executes if $T(t) > 0.90$ (SECURITY GATE)

Result: ~20 high-trust sessions needed to significantly shift baseline.
Attacker cannot corrupt identity during a takeover.

9. Security & Production Hardening

What We Implemented

Security Layer	Implementation	Details
CORS Restriction	Allowlisted origins only	localhost, Firebase, Render — not wildcard
HTTP Rate Limiting	Token bucket (50 req/s, burst 100)	Per-IP, returns 429 with Retry-After
WebSocket Rate Limiting	Per-user token bucket (5 evt/s, burst 10)	Prevents MiniLM inference flooding
Input Validation	Pydantic with bounds checking	All 16 features have min/max constraints
Auth System	HMAC-SHA256 tokens, PBKDF2 passwords	100K iterations, 24h token expiry
Pipeline State Persistence	Redis-backed (every 5 events)	Sessions survive server restarts
Isolation Forest Pre-seeding	100 synthetic normal samples at init	No cold-start vulnerability window
Anti-Poisoning Gate	Baseline only updates if $T > 0.90$	Attackers cannot corrupt user identity
Audit Trail	JSON Lines per day, every decision logged	Full compliance traceability
Graceful Degradation	Redis/Postgres failures → in-memory fallback	Never blocks users due to infra issues

Blocked Session Enforcement	Once BLOCK fires, all further events rejected	Prevents bypass attempts
Single Pipeline (no duplication)	SessionManager → thin facade over EventProcessor	No maintenance divergence risk

Security Architecture Decision: Why Not JWT?

We use HMAC-signed tokens instead of JWT because: 1. No external library dependency for verification 2. Server-side validation is trivial (recompute hash, compare) 3. Simpler than JWT for a hackathon with equivalent security 4. Token structure: `{user_id}.{username}.{timestamp}.{hmac_signature}`

10. Attack Scenarios & Detection

Scenario 1: Normal User (Baseline)

Pattern: Consistent typing (~3.5 cps), low hesitation, stable gyroscope, natural rhythm variance.

Trust trajectory: $T \in [0.95, 0.99]$ — completely flat, stable.

Cognitive state: calm/focused.

Decision: ALLOW throughout.

Scenario 2: Scam Call Victim (Social Engineering)

What happens: 1. User receives fraudulent call ("Your account has been compromised, transfer funds to this safe account") 2. Stress escalates: hesitation increases, correction rate spikes, typing slows 3. Gyroscope shows hand tremor, touch duration extends (freezing) 4. User attempts high-value transfer to unknown beneficiary

Trust trajectory: T: 0.95 → 0.78 → 0.55 → 0.38 (progressive collapse)

Detection signals: - Cognitive state escalates: focused → distressed → panicked → coerced - CUSUM fires at step 4-6 (accumulated behavioral drift) - Transaction scorer flags: high amount + new beneficiary + distressed state - Trust velocity goes deeply negative ($dT/dt < -0.05$) - Entropy elevated (oscillating, uncertain behavior)

Decision: STEP_UP at distressed → BLOCK at panicked + high-value

Scenario 3: Remote Access Malware (Bot/RAT)

What happens: 1. AnyDesk/TeamViewer is installed, attacker takes remote control 2. Input becomes perfectly rhythmic, unnaturally fast, zero hesitation 3. Swipe paths are perfectly straight (programmatic) 4. Gyroscope shows zero variance (device on table, not in hand) 5. Attacker initiates max-value transfer to unknown account

Trust trajectory: T: 0.95 → 0.25 (instant collapse within 2-3 events)

Detection signals: - Cognitive state: robotic (immediately) - Typing rhythm variance < 5.0 (machine-level uniformity) - Typing speed > 9 cps (exceeding human capability) - Swipe straightness > 0.97 (programmatic paths) - Gyroscope < 0.002 (device placed on surface) - CUSUM instant-jump fires (single-step drop > 0.10) - Isolation Forest flags extreme anomaly

Decision: BLOCK within 4-6 seconds (2-3 heartbeats)

Scenario 4: Gradual Account Takeover

What happens: 1. Credentials leaked, different person logs in on registered device 2. Behavior is different but not extreme (different typing style, different swipe patterns) 3. No distress signals — the attacker is calm and deliberate

Trust trajectory: T: 0.99 → 0.87 → 0.82 → 0.78 (slow drift over 20+ events)

Detection signals: - Behavioral similarity drops to 0.75-0.85 range (not catastrophic, but sustained) - CUSUM accumulates evidence over time (gradual drift detection) - Cognitive state stays focused/calm (no distress) - Entropy is low (consistent but different behavior)

Decision: STEP_UP after ~10 events (requires verification, doesn't hard-block)

11. Tech Stack

Layer	Technology	Version	Purpose
Backend Framework	FastAPI	Latest	Async REST + WebSocket server
ML Embeddings	sentence-transformers (MiniLM-L6-v2)	Latest	384-dim behavioral fingerprints
Cognitive Classifier	scikit-learn (Random Forest)	Latest	6-state mental state classification
Anomaly Detection	scikit-learn (Isolation Forest)	Latest	Unsupervised zero-day anomaly scoring
Drift Detection	Custom CUSUM	N/A	Statistical change-point detection
Validation	Pydantic v2	Latest	Request/response schema enforcement

Database	PostgreSQL	16	Persistent storage (7 tables)
Cache/State	Redis	7	Session state, pipeline recovery, alerts
Dashboard Framework	React	18	Concurrent rendering, suspense
Build Tool	Vite	6.3	Instant HMR, optimized production builds
Styling	Tailwind CSS	4.1	Utility-first responsive design
Animation	Motion (Framer)	12	Smooth transitions and micro-interactions
Charts	Recharts	2.15	Real-time trust visualization
3D Effects	Three.js	0.184	Shader-based landing page effects
SDK Language	TypeScript	5.4	Type-safe client library
Containerization	Docker Compose	Latest	Full-stack orchestration
Hosting (Backend)	Render	Free tier	Auto-deploy from GitHub
Hosting (Dashboard)	Firebase Hosting	Latest	Global CDN, SSL
Python	3.11	3.11	Backend runtime
Node.js	18+	18+	Dashboard tooling

12. API Reference

Authentication

Method	Endpoint	Description
POST	/api/v1/auth/register	Create account (username, email, password)
POST	/api/v1/auth/login	Get auth token
POST	/api/v1/auth/verify?token=	Validate token

GET	/api/v1/auth/me?token=	Get current user info
-----	------------------------	-----------------------

Session Management

Method	Endpoint	Description
POST	/api/v1/session/start	Start behavioral monitoring session
POST	/api/v1/session/end	End session, get summary

Behavioral Events

Method	Endpoint	Description
POST	/api/v1/event	Submit behavioral event (HTTP fallback)

Monitoring (Dashboard)

Method	Endpoint	Description
GET	/api/v1/session/{user_id}	Session status + trust history
GET	/api/v1/session/{user_id}/history	Trust score timeline
GET	/api/v1/session/{user_id}/alerts	Session alerts
GET	/api/v1/sessions	List all active sessions

Audit & Explainability

Method	Endpoint	Description
POST	/api/v1/audit/explain	Get decision explanation with root causes
GET	/api/v1/audit/session/{user_id}/summary	Full session audit summary

WebSocket Endpoints

Protocol	Endpoint	Description
WS	/ws/{user_id}	SDK real-time connection (send events, receive decisions)
WS	/ws/dashboard	Dashboard broadcast channel (receive all trust updates)

Health

Method	Endpoint	Description
GET	/	Health check
GET	/status	System status + cache health + active connections
GET	/metrics	Pipeline latency p50/p95/p99, decision counters

13. Deployment

Local Development

```
# Backend
cd AEGIS_X
python -m venv venv
venv\Scripts\activate          # Windows
pip install -r requirements/base.txt

# Train cognitive model (first time only)
python scripts/generate_cognitive_dataset.py
python scripts/train_cognitive_model.py

# Start backend (port 8000)
uvicorn backend.main:app --reload --port 8000

# Dashboard (separate terminal)
cd dashboard
npm install
npm run dev                    # Runs on port 3000 with proxy to :8000
```

Docker (Full Stack)

```
docker-compose up --build
# Backend:    http://localhost:8000
# Dashboard:  http://localhost:3000
# Swagger:    http://localhost:8000/docs
# Postgres:   localhost:5432
# Redis:      localhost:6379
```

Production (Current Deployment)

- **Backend:** Render (auto-deploy) → <https://aegisx-backend-t1v7.onrender.com>
- **Dashboard:** Firebase Hosting → <https://aegisx-2026.web.app>
- **Database:** Render PostgreSQL (managed)
- **Cache:** Render Redis (managed)

Environment Variables

```
# Database
POSTGRES_USER=postgres
POSTGRES_PASSWORD=<secure>
POSTGRES_DB=aegisx
POSTGRES_HOST=<host>
POSTGRES_PORT=5432

# Cache
REDIS_HOST=<host>
REDIS_PORT=6379
REDIS_DB=0

# Security
AEGISX_SESSION_SECRET=<random-64-chars>
AEGISX_CORS_ORIGINS=https://aegisx-2026.web.app,https://your-app.com
AEGISX_WS_RATE_LIMIT=5
AEGISX_WS_RATE_BURST=10

# Render
RENDER_EXTERNAL_URL=https://aegisx-backend-t1v7.onrender.com
```

14. What We Built — Complete Feature List

ML & AI (7 models/algorithms)

- [x] MiniLM-L6-v2 behavioral embedding engine (384-dim semantic fingerprints)
- [x] Random Forest cognitive state classifier (6 states, 8 features)
- [x] Isolation Forest unsupervised anomaly detector (pre-seeded, online learning)
- [x] CUSUM statistical change-point detection (dual: cumulative + instant-jump)
- [x] Exponential Moving Average baseline adaptation (with anti-poisoning gate)
- [x] Multi-signal trust fusion formula (4-component weighted composite)
- [x] Fraud intent prediction (4-vector: coercion/takeover/robotic/anomaly)

Backend Services (18 services)

- [x] Trust Pipeline orchestrator (11-step, <100ms)

- [x] Event Processor (central controller with alerts + audit + persistence)
- [x] Feature Engineering (16-dim extraction with validation + imputation)
- [x] Behavioral Serializer (numeric → semantic text for MiniLM)
- [x] Embedding Service (singleton MiniLM, L2-normalized)
- [x] Baseline Service (enrollment, EMA update, save/load, confidence scoring)
- [x] Similarity Service (cosine similarity + risk classification)
- [x] History Service (sliding window + velocity + acceleration + entropy)
- [x] Drift Service (CUSUM detector with severity classification)
- [x] Anomaly Service (Isolation Forest, cold-start eliminated)
- [x] Cognitive Service (Random Forest + trajectory analysis)
- [x] Trust Service (T(t) computation + TransactionScorer + DeviceScorer)
- [x] Decision Service (ALLOW/STEP_UP/BLOCK with 6 escalation rules)
- [x] Fraud Predictor (intent vectors + trajectory classification)

- [x] Risk Service (unified risk aggregation)
- [x] Explanation Service (incident narratives + root cause)
- [x] Cache Service (Redis with graceful in-memory fallback)
- [x] Session Manager (thin facade, backward-compatible)

API Layer (14 endpoints + 2 WebSocket)

- [x] Authentication (register, login, verify, me)
- [x] Session lifecycle (start, end)
- [x] Behavioral event submission (HTTP)
- [x] Monitoring (status, history, alerts, active sessions)
- [x] Audit (explain decision, session summary)
- [x] Health (status, metrics)
- [x] WebSocket SDK channel (per-user, bidirectional)

- [x] WebSocket Dashboard broadcast (all decisions streamed)

Security (8 layers)

- [x] CORS restriction (allowlisted origins, not wildcard)
- [x] HTTP rate limiting (token bucket, 50 req/s)
- [x] WebSocket rate limiting (per-user, 5 evt/s + burst 10)
- [x] Input validation (Pydantic bounds on all 16 features)
- [x] Auth system (PBKDF2 passwords, HMAC tokens, 24h expiry)
- [x] Blocked session enforcement (reject all events after BLOCK)
- [x] Anti-poisoning baseline gate (update only when $T > 0.90$)
- [x] Pipeline state persistence (crash recovery via Redis)

Dashboard (9 pages)

- [x] Animated landing page (CardSwap, BentoGrid, FlipWords, RippleGrid, ShaderRipple)

- [x] Live Demo page (3 attack simulators with real-time WebSocket)
- [x] Live Monitor (trust gauge, decision feed)
- [x] Trust Timeline (area chart, temporal dynamics overlay)
- [x] Cognitive Analysis (state machine, stability tracking)
- [x] Incident Explorer (root cause, intent vectors, recommendations)
- [x] Session Replay (attack timeline, event-by-event audit)
- [x] Login/Register (with auth guard)
- [x] Protected layout with sidebar navigation

SDK (TypeScript)

- [x] 4 behavioral collectors (typing, touch, navigation, hesitation)
- [x] Feature extractor (16-dim aggregation every 2s)
- [x] WebSocket transport (auto-reconnect, heartbeat)

- [x] IndexedDB event buffer (offline resilience)
- [x] Browser-specific bindings
- [x] Public API: init, start, stop, onDecision

Simulators (3 attack scenarios)

- [x] Normal User simulator (calm browsing, small transactions)
- [x] Scam Victim simulator (escalating coercion, high-value transfer)
- [x] Malware Bot simulator (robotic precision, instant detection)

Data Pipeline & Training

- [x] Synthetic behavioral data generator (10,500 samples)
- [x] Cognitive dataset generator (12,000 labeled samples)
- [x] Cognitive model training script (Random Forest)
- [x] Database schema initialization script

Infrastructure

- [x] Docker Compose (full stack orchestration)
- [x] Dockerfile (backend container)
- [x] Firebase hosting config (dashboard)
- [x] Render deployment (backend)
- [x] Environment configuration (dev + production)

Total Implementation

Category	Count
Python backend services	18
ML models/algorithms	7
API endpoints	16
Dashboard pages	9
SDK collectors	4
Security layers	8
Attack simulators	3
Training scripts	4
Infrastructure configs	5

Total files authored: ~60+ source files across backend, dashboard, SDK, and scripts.

Built for DFS & IBA Cyber Security PSBs Hackathon 2026 Central Bank of India × MNNIT Allahabad